CS-2710-1 Spring 2026

Computer Organization and Systems (COS)

Lecture #19

15 April 2026

Ashoka University

BBB

Cache Memory

First introduced in a landmark paper:

M. Wilkes, Slave memories and dynamic storage allocation, *IEEE Transactions on Electronic Computers*, pp. 270-271, 1965

First commercially implemented in IBM 360/85 (1968)

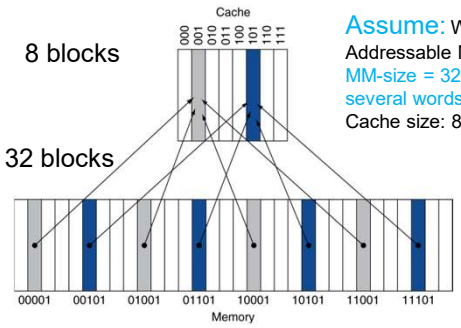
More than ~1000 research papers published since then …

Foundational concept in CA, OS, Networks, ..

What it means? Cache is a high-speed, temporary data storage layer that keeps a subset of frequently accessed data (so that future requests from CPU can be served faster ..); based on temporal/spatial locality;

Direct-Mapped Cache

- Location in cache determined by MM-address
- Direct mapped: MM → cache, is fixed, i.e., the destination in cache is unique;
  - (Block address) modulo (#Blocks in cache) Many-to-one mapping



Assume: Width of address bus: 5 bits
Addressable Memory Space = 2^5 = 32 blocks
MM-size = 32 blocks (a block might comprise several words);
Cache size: 8 blocks

- #Blocks is a power of 2
- Use low-order address bits;
- CPU generates MM-address;
- How it is converted to cache address (index)?

Does a cache entry contain garbage?

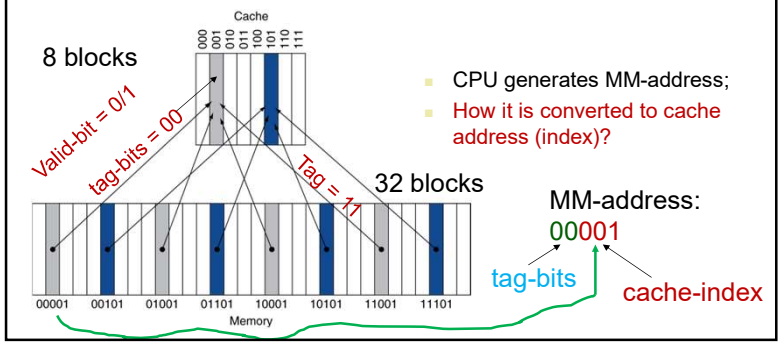
- Problem: when a program starts up, cache is empty (garbage).
  - It might contain data left from previous program/random bits;
  - How do you make sure you don't match a tag accidentally?
- Solution:
- An extra "valid" bit per cache-entry

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
  - Store missing part of block address as well as the data (cache data means instructions/data)
  - Actually, only need the high-order bits
  - Called the tag (resolves the problem of many-to-one mapping: MM→ Cache)
- What if there is no data in a location?
  - Valid bit: 1 = present, 0 = not present
  - Initially all valid-bits set to 0; a valid-bit is set to 1, when the requested item is loaded from MM to cache

Direct-Mapped Cache

- Location in cache determined by MM-address
- Direct mapped: only one choice
  - (Block address) modulo (#Blocks in cache)



- CPU generates MM-address;
- How it is converted to cache address (index)?

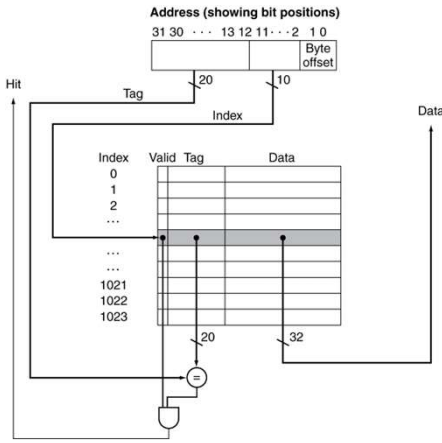
Cache Management

- Block placement in cache from MM?
  (i) direct-mapped, (ii) set-associative, (iii) fully-associative;
- Block identification in cache by CPU?
  index bits, tag-bits, valid-bit;
- On cache miss -> block replacement?
  LRU, LFU, random;
- Write Strategy?
  write through, write back (need dirty-bit);
- Miss handling?
  read miss; write miss;

MM-Address to Cache-Address Resolution (Block size = 1 word)

Hardware realization

MM-address – 32 bits;
Cache: 1 K blocks;
1 block = 1 word
Direct-mapped, write through;
What will be the cache size?



rows (cache-lines) = 1024;
Each row comprises
1 (valid-bit) + 20 (tag-bits) + 32 (data-bits) = 53 bits;
Write-through does not need any extra bit;
Total size of the cache
= 53 ×1024 bits = 53 Kbits

Cache Hit/Miss

- CPU sends address for MM requesting retrieval (instruction/data);
- MM-address resolved to cache-address; item first searched in cache;
- On cache hit -> read-hit, write-hit (actions?)
- On cache miss -> read-miss, write-miss (actions?)
  - Stall the CPU pipeline
  - Fetch block from next level of hierarchy
  - Instruction cache miss
    - PC = PC -4; Restart instruction fetch: PC = PC +4
  - Data cache miss
    - Complete data access

Issues in Cache Management

- Block placement: MM → Cache
  - direct-mapped, set-associative, fully associative
- Block identification: index, tag, valid-bit
- Block replacement: LRU, LFU, random
- Write strategy (CPU wants to store data in Mem): write-through, write-back (dirty-bit)
- Miss handling:
  - -- cache read miss
  - --- cache write miss
    - * write-allocate, no-write-allocate

Memory Hierarchy Basics

- When a word is not found in the cache, a *miss* occurs:
  - Fetch word from lower level in hierarchy, requiring a higher latency reference
  - Lower level may be another cache or the main memory
  - Also fetch the other words contained within the *block*
    - Takes advantage of spatial locality
  - Place block into cache in any location within its *set*, determined by address
    - [block address (MM)] *mod* (number of sets)

Memory Hierarchy Basics

- Hit-time (time to match a requested item in cache and if found, to transfer it to CPU); time measured in clock cycles;
- Miss rate
  - Fraction of cache requests that result in misses
- Miss-penalty (time to pull an item to cache from the lower level and to transfer it to CPU)
- Causes of misses
  - Compulsory
    - First reference to a block
  - Capacity
    - Blocks discarded and later retrieved
  - Conflict
    - Program makes repeated references to multiple addresses from different blocks that map to the same location in the cache

Parameters in Cache Management

Average memory access time (AMAT)
= hit-time + (miss-rate × miss-penalty)
Note: hit-time and miss-penalty are expressed in # clock cycles

Average # memory-stall cycles (AMSC) =
(# memory requests/IC) × miss-rate × miss-penalty

CPU-time = IC × CPI × CCT
= IC × (Ideal-CPI + AMSC) × CCT

Ideal-CPI refers to the case when we have perfect cache, i.e., without any misses

Dealing with Stores (CPU writing data to cache)

- Stores must be handled differently than loads, because...
  - they don't necessarily require the CPU to stall
  - they change the content of cache

    Creates a memory consistency question ... how do you ensure memory gets the correct value?

    the one in cache

Policy decisions for stores (writes)

- Do you keep memory and cache identical?
  - write-through cache: all writes go to both cache and main memory
  - write-back cache: writes go only to cache Modified cache lines are written back to memory when the line is replaced/or when the program ends
- Do you make room in cache for store miss?
  - write-allocate: on a store miss, bring target line into the cache.
  - No-write-allocate: on a store miss, ignore cache

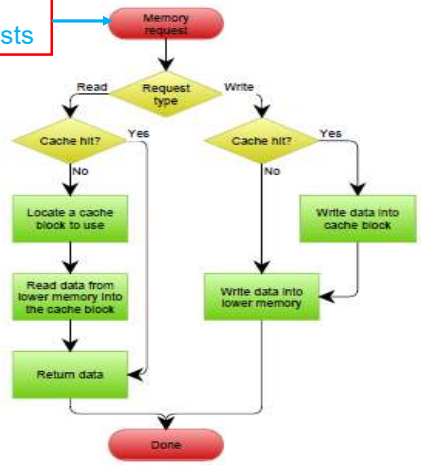
Dealing with stores

- On a store hit, write the new data to cache
  - In a *write-through* cache, write the data immediately to memory.
  - In a *write-back* cache
    - Mark the line as dirty.
      - means cache has correct value, but memory doesn't
    - On any cache miss in a write-back cache, if the line to be replaced in the cache is dirty, write it back to memory
    - Note: Read miss in cache might cause "write" to memory!
- On a store miss,
  - In a *write-allocate* cache,
    - Initiate a cache block load from memory
  - In a no-write-allocate cache,
    - Write directly to memory

Solution:
write buffer olds data waiting to be written to memory CPU continues immediately;
Only stalls on write if write buffer is already full

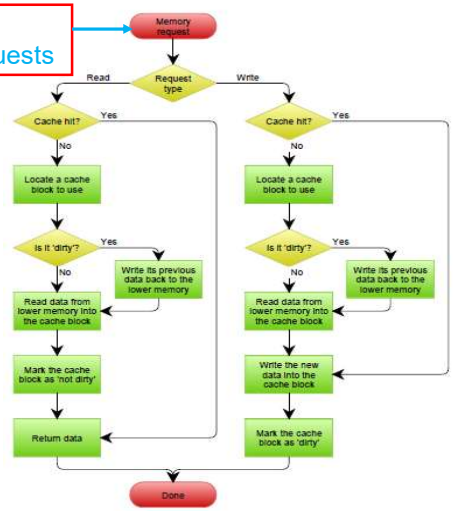
Write-Through with No-Write-allocation

CPU sends Memory requests



Write-Back with Write-Allocation

CPU sends Memory requests

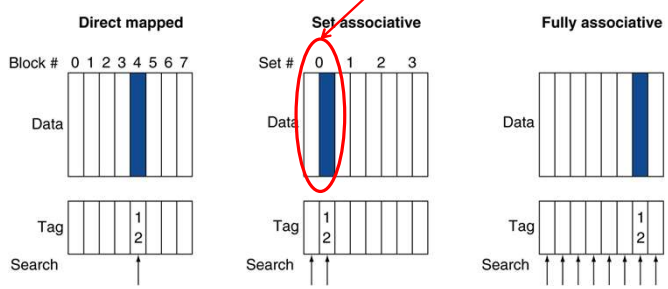


Cache Architecture other than Direct-Mapped: Associative Caches

- *n*-way set associative
- (direct-mapped is 1-way set associative)
  - Each set contains *n* entries
  - *MM-address, n, cache size,* determine to which set element to go ..
  - Search all entries in a given set at once
  - *n* comparators
  - Fully associative
  - Allow a given block to go in any cache entry
  - Requires all entries to be searched at once:
  - hit-time increases, or needs parallel hardware

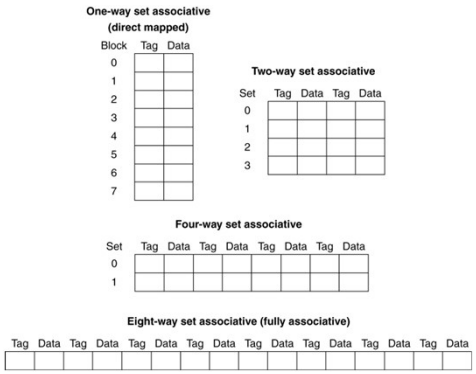
Associative Cache Example

Size of each set = 2
2-way set associative



Spectrum of Associativity

- For a cache with 8 entries



Associative Cache: Example

- 5-bit MM-address
- # cache blocks = 4, 1 word/block

MM address requests: 0, 8, 0, 6, 8, 6

Find the number of misses for the following three cache-architectures:

(i) direct-mapped → # misses = 5

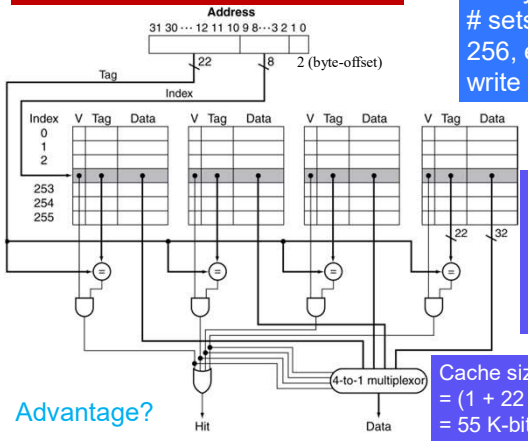
(ii) 2-way set-associative → # misses = 4

(iii) fully-associative → # misses = 3

Set-Associative Cache Organization: Example 1

Cache index = MM-address *mod* 256

MM-address – 32 bits;
Cache: 1 K block;
1 block = 1 word (32 bits)
4-way set-associative;
sets (# cache rows) = 256, each set size = 4;
write through



tag-bits = 22

Associativity reduces miss-rate (by reducing conflict misses), but may increase hit-time;
Typical designs:
8-way set associative

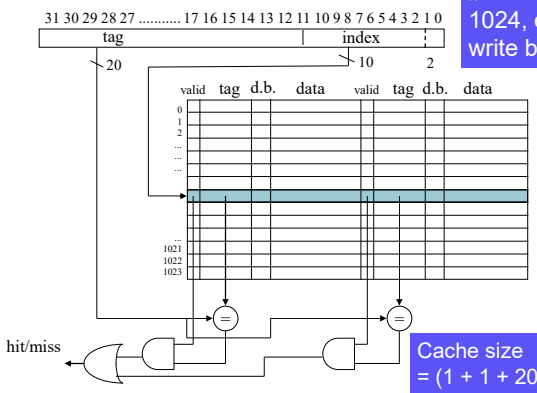
Cache size
= (1 + 22 + 32) × 4 × 256 bits
= 55 K-bits

Advantage?

Example 2: Set-Associative Cache

Cache index = MM-address *mod* 1024

MM-address – 32 bits;
Cache: 2 K blocks;
1 block = 1 word (32 bits)
2-way set-associative;
sets (# cache rows) = 1024, each set size = 2;
write back



d.b. ⇒ (dirty-bit)

Cache size
= (1 + 1 + 20 + 32) × 2 × 1024 bits
= 108 K-bits

Replacement (Eviction) Policy

- Direct mapped: no choice
- Set associative
  - Prefer non-valid entry, if there is one
  - Otherwise, choose among entries in the set
- Least-recently used (LRU)
  - Choose the one unused for the longest time
    - Simple for 2-way, manageable for 4-way, too hard beyond that
- Random
  - Gives approximately the same performance as LRU for high-associativity

Cache Performance: Example

- Given
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty = 100 cycles
  - Base CPI (ideal cache) = 2
  - Load & stores are 36% of instructions
- Miss cycles per instruction
  - I-cache: 0.02 × 100 = 2
  - D-cache: 0.36 × 0.04 × 100 = 1.44
- Actual CPI = 2 + 2 + 1.44 = 5.44
  - Ideal CPU is 5.44/2 =2.72 times faster

Average Access Time

- Hit time is also important for performance
- Average memory access time (AMAT)
  - AMAT = Hit time + (Miss rate × Miss penalty)
- Example
  - CPU with 1ns clock, hit time = 1 cycle, miss penalty = 20 cycles, I-cache miss rate = 5%
  - AMAT = 1 + 0.05 × 20 = 2ns
    - 2 cycles per instruction

Cache: Key Points

- Caches give illusion of a large, cheap memory with the access time of a fast, expensive memory
- Caches take advantage of temporal locality and spatial locality
- Cache design presents many options (block size, cache size, associativity) that an architect must combine to minimize miss rate and access time, so as to maximize performance